

O projeto GranaFlow possui uma estrutura MVC básica funcional, mas apresenta alguns erros críticos e oportunidades de melhoria que, se abordados, aumentarão significativamente sua robustez, segurança e manutenibilidade. A correção das inconsistências do banco de dados e da duplicação de métodos, juntamente com a implementação consistente da proteção CSRF, são passos essenciais para estabilizar a aplicação. As melhorias sugeridas em refatoração e otimização contribuirão para um código mais limpo e performático a longo prazo. Recomenda-se priorizar as correções dos erros críticos antes de implementar as melhorias gerais.

## Introdução

Este documento detalha uma análise do projeto GranaFlow, identificando **erros críticos** que afetam a funcionalidade e a segurança, bem como **oportunidades de melhoria** para otimizar o código, a manutenibilidade e a experiência do usuário. O objetivo é fornecer um guia claro para aprimorar a aplicação, garantindo sua estabilidade e escalabilidade.

## Erros Críticos e Resoluções

### 1. Inconsistências no Schema do Banco de Dados

**Problema:** O arquivo `sql/banco.sql` fornecido inicialmente está incompleto e não reflete a estrutura de banco de dados esperada pelo código da aplicação. Tabelas essenciais como `salarios`, `gastos_recorrentes` e `dinheiro_guardado`, além de colunas como `tipo` e `valor_guardado` na tabela `metas`, estão ausentes. Isso causa erros de tempo de execução (runtime errors) quando o código tenta acessar essas estruturas inexistentes, especialmente nas funcionalidades de dashboard, metas e gastos recorrentes.

#### Detalhes:

- **Tabelas Ausentes:** `salarios`, `gastos_recorrentes`, `dinheiro_guardado`.
- **Colunas Ausentes em `metas`:** `tipo` (para diferenciar metas de gasto e reserva) e `valor_guardado` (para rastrear o progresso das metas de reserva).

**Resolução:** Utilize o arquivo `sql/banco_completo.sql` para criar ou atualizar o banco de dados. Este arquivo contém todas as tabelas e colunas necessárias para o funcionamento correto da aplicação. Recomenda-se executar este script em um ambiente de desenvolvimento para garantir que todas as dependências do banco de dados sejam atendidas antes de qualquer implantação em produção.

#### Exemplo de Correção (SQL):

```
-- Exemplo de criação da tabela 'salarios' que estava faltando
CREATE TABLE IF NOT EXISTS salarios (
  id INT AUTO_INCREMENT PRIMARY KEY,
  id_usuario INT NOT NULL,
  valor DECIMAL(10,2) NOT NULL DEFAULT 0.00,
  data_atualizacao DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  FOREIGN KEY (id_usuario) REFERENCES usuarios(id_usuario) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- Exemplo de alteração da tabela 'metas' para adicionar colunas 'tipo' e
ALTER TABLE metas
ADD COLUMN valor_guardado DECIMAL(10,2) DEFAULT 0.00 AFTER valor_limite,
ADD COLUMN tipo ENUM('\gasto\','reserva\') DEFAULT '\gasto\' AFTER valor_guardado;
```

## 2. Duplicação de Métodos `salvar()` em `MetasController.php`

**Problema:** O controlador `app/Controllers/MetasController.php` possui duas definições para o método `public function salvar()`. Em PHP, isso resulta em um erro fatal (`Cannot redeclare MetasController::salvar()`), impedindo que a classe seja carregada e que qualquer funcionalidade relacionada a metas funcione corretamente. Uma das implementações parece ser para submissões de formulário tradicionais (com validação CSRF e campo `tipo`), enquanto a outra é para requisições AJAX (sem validação CSRF e sem o campo `tipo`).

**Detalhes:**

- **Primeiro `salvar()`** : Linhas 71-130, inclui validação CSRF e o campo `tipo`.
- **Segundo `salvar()`** : Linhas 288-335, não inclui validação CSRF e omite o campo `tipo`.

**Resolução:** Os dois métodos `salvar()` devem ser unificados em uma única função que trate ambas as situações (submissão de formulário tradicional e requisições AJAX) de forma consistente. A validação CSRF deve ser aplicada a todas as submissões POST, independentemente de serem AJAX ou não, para garantir a segurança. O campo `tipo` também deve ser tratado de forma consistente.

**Exemplo de Correção (PHP - Lógica):**

```
// Em app/Controllers/MetasController.php

public function salvar()
{
    if ($_SERVER['REQUEST_METHOD'] !== 'POST') {
        redirecionar('metas');
    }

    $id_usuario = $_SESSION['id_usuario'];
    $isAjax = $this->isAjax(); // Método auxiliar para verificar se é AJAX

    // Validação CSRF (aplicar sempre que houver submissão POST)
    if (!$isAjax && (empty($_POST['csrf_token']) || !$this->validarTokenC)) {
        $_SESSION['erro'] = "Solicitação inválida.";
        redirecionar('metas');
    }

    $id_meta = intval($_POST['id_meta'] ?? 0);
    $nome = $this->sanitizar($_POST['nome_meta'] ?? '');
    $valor = floatval(str_replace(',', '.', $_POST['valor_limite'] ?? 0));
    $tipo = $this->sanitizar($_POST['tipo'] ?? 'gasto'); // Garantir que é

    // ... (restante das validações e lógica de salvar/atualizar)

    if ($sucesso) {
        if ($isAjax) {
            $this->jsonResponse(true, $mensagem, $this->getDashData($id_u
        ) else {
            $_SESSION['sucesso'] = $mensagem;
        }
    } else {
        if ($isAjax) {
            $this->jsonResponse(false, 'Erro ao salvar meta.');
        } else {
            $_SESSION['erro'] = "Erro ao salvar meta.";
        }
    }

    if (!$isAjax) {
        redirecionar('metas');
    }
}
```

### 3. Falta de Proteção CSRF em Formulários AJAX do Dashboard

**Problema:** Os formulários AJAX presentes na view `app/Views/dashboard/index.php` (como os de adicionar gasto, salvar salário, guardar dinheiro em meta, etc.) utilizam a classe `form-ajax` para submissão assíncrona, mas não incluem um campo oculto `csrf_token`. Isso os torna vulneráveis a ataques de Cross-Site Request Forgery (CSRF), onde um atacante pode forçar o navegador de um usuário autenticado a enviar requisições indesejadas à aplicação.

#### Detalhes:

- Formulários como os de salário (linhas 214-223), guardar dinheiro avulso (244-269), adicionar gasto (287+), recorrente (329+) e meta (384+) na `dashboard/index.php` não possuem o campo `csrf_token`.
- O `GastosController.php` (linhas 100-104) e `MetasController.php` (linhas 79-83) já possuem a lógica de validação CSRF para formulários tradicionais, mas essa validação não é acionada ou é ignorada para requisições AJAX que não enviam o token.

**Resolução:** É fundamental incluir o campo `csrf_token` em todos os formulários AJAX que realizam operações de escrita (POST, PUT, DELETE). O token deve ser gerado no servidor (pelo `Controller` base) e incluído na view. No lado do cliente (JavaScript), o token deve ser adicionado ao `FormData` antes da submissão AJAX. No servidor, a validação do token deve ser realizada para todas as requisições POST, independentemente de serem AJAX ou não.

#### Exemplo de Correção (HTML/PHP na View):

```
<!-- Em app/Views/dashboard/index.php, dentro de cada formulário .form-ajax -->
<form method="POST" action="..." class="form-ajax">
  <input type="hidden" name="csrf_token" value="<?= $csrf_token ?? ' ' ?:" />
  <!-- ... outros campos do formulário ... -->
</form>
```

#### Exemplo de Correção (JavaScript em `public/js/dashboard-ajax.js`):

```
// Em public/js/dashboard-ajax.js, dentro de submitForm(form)

// ...
const formData = new FormData(form);
formData.append("_ajax", "1");

// Adicionar o token CSRF se ele existir na página (pode ser um meta tag)
const csrfTokenElement = document.querySelector('meta[name="csrf-token"]');
if (csrfTokenElement) {
    formData.append('csrf_token', csrfTokenElement.content || csrfTokenElement.textContent);
}
// ...
```

## 4. HTML Malformado no Dashboard ( dashboard/index.php )

**Problema:** A view `app/Views/dashboard/index.php` contém um erro de sintaxe HTML na tag `<body>`. A linha `<body data-base-url="<?= BASE_URL ?>"` está faltando o caractere de fechamento `>` antes de um comentário HTML. Isso pode levar a problemas de renderização no navegador, dificultar a interpretação do DOM e, potencialmente, impedir que scripts JavaScript acessem corretamente o atributo `data-base-url`.

### Detalhes:

- Linha 20 de `app/Views/dashboard/index.php`.

**Resolução:** Adicionar o caractere `>` para fechar corretamente a tag `<body>`.

### Exemplo de Correção (HTML):

```

<!-- Antes: -->
<body data-base-url="<?= BASE_URL ?>"
  <!-- Navbar -->

<!-- Depois: -->
<body data-base-url="<?= BASE_URL ?>">
  <!-- Navbar -->
```

# Melhorias Sugeridas

## 1. Refatoração de Controladores e Modelos

**Oportunidade:** Observa-se alguma duplicação de lógica e responsabilidades em controladores como `GastosController.php` e `MetasController.php`, especialmente no tratamento de requisições AJAX versus requisições tradicionais. Além disso, a lógica de `getDashData()` é repetida em múltiplos controladores.

### Sugestão:

- **Unificar Lógica de Salvar/Atualizar:** Consolidar métodos como `salvar()` e `adicionar()` em uma única função que possa ser reutilizada, diferenciando a ação (criar ou atualizar) com base na presença de um ID.
- **Serviços/Helpers Compartilhados:** Criar classes de serviço ou funções auxiliares (helpers) para lógica comum, como a preparação de dados do dashboard ( `getDashData()` ), que atualmente é duplicada. Isso reduziria a repetição de código e facilitaria a manutenção.
- **Separação de Responsabilidades:** Garantir que os modelos ( `Model` ) sejam responsáveis apenas pela interação com o banco de dados, e os controladores ( `Controller` ) pela orquestração da lógica de negócio e preparação dos dados para as views.

## 2. Tratamento de Erros Aprimorado

**Oportunidade:** O tratamento de erros atual utiliza `$_SESSION['erro']` e `redirecionar()`, ou `jsonResponse()` para AJAX. Embora funcional, pode ser aprimorado para oferecer mais detalhes aos desenvolvedores e uma experiência mais fluida aos usuários.

### Sugestão:

- **Logging de Erros:** Implementar um sistema de logging robusto para registrar erros internos da aplicação (erros de banco de dados, exceções não tratadas) em arquivos de log, em vez de apenas exibi-los ao usuário ou redirecionar. Isso é crucial para depuração em produção.
- **Páginas de Erro Amigáveis:** Criar páginas de erro personalizadas (e.g., 404 Not Found, 500 Internal Server Error) para melhorar a experiência do usuário em caso de falhas inesperadas.
- **Validação de Entrada no Servidor:** Embora já exista, a validação pode ser mais granular e retornar mensagens de erro específicas para cada campo, especialmente em requisições AJAX, para que o frontend possa exibir feedback em tempo real ao lado dos campos inválidos.

### 3. Segurança (Além de CSRF)

**Oportunidade:** A segurança é um aspecto contínuo no desenvolvimento web. Além da correção do CSRF, outras áreas podem ser revisadas.

**Sugestão:**

- **Prepared Statements:** O uso de `mysqli::prepare()` e `bind_param()` já está presente em `Model.php`, o que é excelente para prevenir SQL Injection. Continuar garantindo que todas as interações com o banco de dados utilizem prepared statements.
- **Sanitização e Validação de Entrada:** Reforçar a sanitização de todas as entradas do usuário e a validação rigorosa dos dados antes de processá-los ou armazená-los. A função `sanitizar()` existente é um bom começo, mas pode ser complementada com validações mais específicas para tipos de dados (e.g., `filter_var()` para e-mails, números).
- **Configuração de Segurança do Servidor:** Garantir que o servidor web (Apache/Nginx) e o PHP estejam configurados com as melhores práticas de segurança (e.g., desabilitar `display_errors` em produção, configurar `open_basedir`, etc.).

### 4. Consistência e Padrões de Código

**Oportunidade:** Manter um código consistente facilita a leitura, a manutenção e a colaboração.

**Sugestão:**

- **Padrões de Nomenclatura:** Adotar um padrão de nomenclatura consistente para variáveis, funções, classes e arquivos (e.g., camelCase para variáveis e funções, PascalCase para classes).
- **Comentários e Documentação:** Adicionar comentários explicativos em blocos de código complexos e documentar a finalidade de cada classe e método, especialmente os públicos. Isso já está presente em algumas partes, mas pode ser estendido.
- **Organização de Código:** Revisar a organização dos arquivos e diretórios para garantir que sigam uma lógica clara e consistente (e.g., todos os helpers em um diretório `app/Helpers`).

## Conclusão

O projeto GranaFlow tem uma base sólida, mas a correção dos **erros críticos** relacionados ao schema do banco de dados, à duplicação de métodos e à proteção CSRF é fundamental para a estabilidade e segurança da aplicação. Uma vez que esses problemas sejam resolvidos, as **melhorias sugeridas** em refatoração, tratamento de erros e padrões de código podem ser implementadas para elevar a qualidade geral do projeto, tornando-o mais robusto, seguro e fácil

de manter. Recomenda-se abordar essas questões de forma iterativa, priorizando os erros mais impactantes primeiro.